Vigenère Cipher with Row & Column Permutations

Rayen Saliba | ID:20231806 | CSC316 | 10/11\2026

1. Overview

This project implements the classical Vigenère cipher and strengthens it with two extra rounds of transposition: a column permutation and a row permutation. The program can both encrypt and decrypt. Every permutation used during encryption is reversed exactly during decryption, so pasting the generated ciphertext back into the program returns the original plaintext.

GitHub repository: https://github.com/RayenSaliba

2. Project Structure

```
project-folder/
|-- vigenere.py     # source code
|-- README.docx     # this documentation
|-- assets/         # screenshots of the program output
    |-- test1.png
    |-- test2.png
    |-- test3.png
```

3. How to Run the Program

The program is written in Python 3 and uses only the standard library, so there is nothing to install and no compilation step.

Requirements

- Python 3.6 or newer

Steps

- Open a terminal in the project folder.
- Run: `python vigenere_Cipher.py` (or `python3 vigenere_Cipher.py` on Linux/macOS).
- Choose the mode: E for Encrypt or D for Decrypt.
- Enter the plaintext (or the ciphertext when decrypting).
- Enter the Vigenère key (letters only).
- Enter the column order, e.g. `3 1 4 2`.
- Enter the row order, e.g. `2 3 1`.

To decrypt, use the same key, column order and row order that were used to encrypt.

Input rules

- Only the letters A to Z are processed. The text is converted to uppercase, and spaces, digits and punctuation are removed.
- A permutation order must use each number from 1 to N exactly once (for example 3 1 4 2 is valid, 3 3 1 2 is not). The program asks again if the input is invalid.

- The text is padded with the letter X so that it fills complete rows and complete row groups. After decryption, a few extra X letters may therefore appear at the end of the message.

4. How the Cipher Works

4.1 Encryption pipeline

Round	Operation	Description
Pre-step	Clean and pad	Keep letters only, make uppercase, pad with X to a multiple of (columns x row-group size).
1	Vigenère substitution	Each letter is shifted by the matching letter of the repeating key (A=0, B=1, ... Z=25).
2	Column permutation	Text is written in rows as wide as the column order; the columns in every row are rearranged by the column order.
3	Row permutation	The rows are taken in groups the size of the row order; the rows inside each group are rearranged by the row order.

4.2 Round 1: Vigenère substitution

For the letter at position `i`: `cipher = (plain + key[i mod len(key)]) mod 26`. Decryption subtracts the key shift instead of adding it.

4.3 Round 2: column permutation

The text is split into rows whose width equals the length of the column order. Using the column order 3 1 4 2 on the row ABCD, the new row is built by taking the 3rd letter, then the 1st, then the 4th, then the 2nd, which gives CADB.

4.4 Round 3: row permutation

The rows are processed in groups of N rows, where N is the length of the row order. With the row order 2 3 1, the rows R1, R2, R3 of each group are rearranged into R2, R3, R1.

4.5 How everything is reversed

Both permutations are handled by two small functions that are exact opposites of each other:

```
def permute(items, order):        # used when encrypting
    return [items[i - 1] for i in order]

def unpermute(items, order):      # used when decrypting
    result = [None] * len(items)
    for position, i in enumerate(order):
        result[i - 1] = items[position]
    return result
```

Because unpermute puts every item back in the position that permute took it from, applying one after the other returns the original data. Decryption simply runs the rounds in reverse order:

```
Encrypt:  plaintext -> Vigenere -> column permute -> row permute -> ciphertext
```

```
Decrypt:  ciphertext -> row undo -> column undo -> Vigenere decrypt -> plaintext
```

The ciphertext is printed as one continuous string of uppercase letters, so it can be copied and pasted directly into the decryption mode.

5. Example Test Cases

Each test case shows a full encryption followed by a successful decryption. The extra X letters at the end of the decrypted text are the padding.

Test Case 1: Basic example

Input	Value
Mode	Encrypt
Plaintext	HELLO WORLD ATTACK AT DAWN
Key	LEMON
Column order	3 1 4 2
Row order	2 3 1

Encryption output:

```
After Vigenere        : SIXZBHSDZQLXFOPVEFRNHRJL
After column permute  : XSZISBDHLZXQPFVORENFJHLR
After row permute     : SBDHLZXQXSZIRENFJHLRPFVO

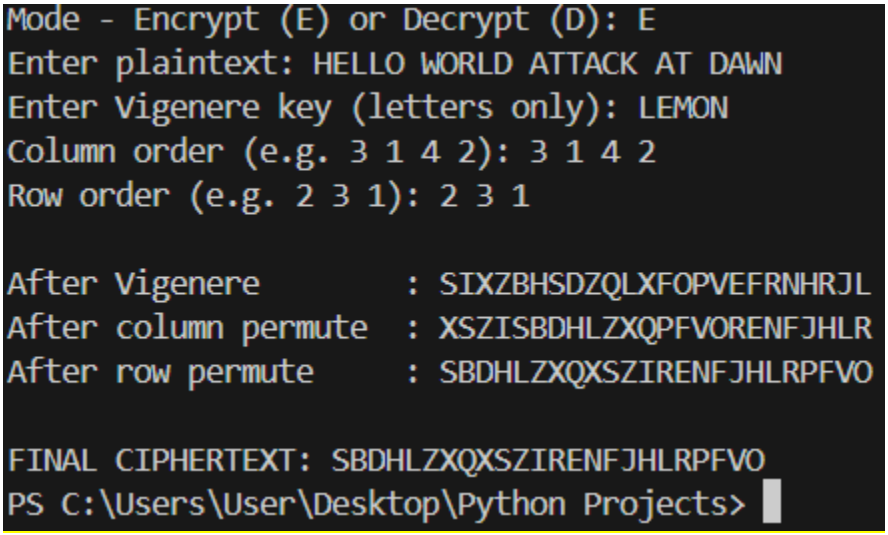
FINAL CIPHERTEXT: SBDHLZXQXSZIRENFJHLRPFVO
```

Decryption output (ciphertext pasted back in):

```
After row undo        : XSZISBDHLZXQPFVORENFJHLR
After column undo     : SIXZBHSDZQLXFOPVEFRNHRJL

DECRYPTED TEXT: HELLOWORLDATTACKATDAWNXX
```

Result: The original message HELLOWORLDATTACKATDAWN is recovered (followed by XX padding).



Test Case 2: Different key and orders

Input	Value
Mode	Encrypt
Plaintext	MEET ME AT THE BRIDGE
Key	CODE
Column order	2 4 1 3
Row order	3 1 2

Encryption output:

```
After Vigenere        : OSHXOSDXVVHFTWGKGLABZLAB
After column permute  : SXOHSXODVFVHWKTGLBGALBZA
After row permute     : VFVHSXOHSXODLBZAWKTGLBGA

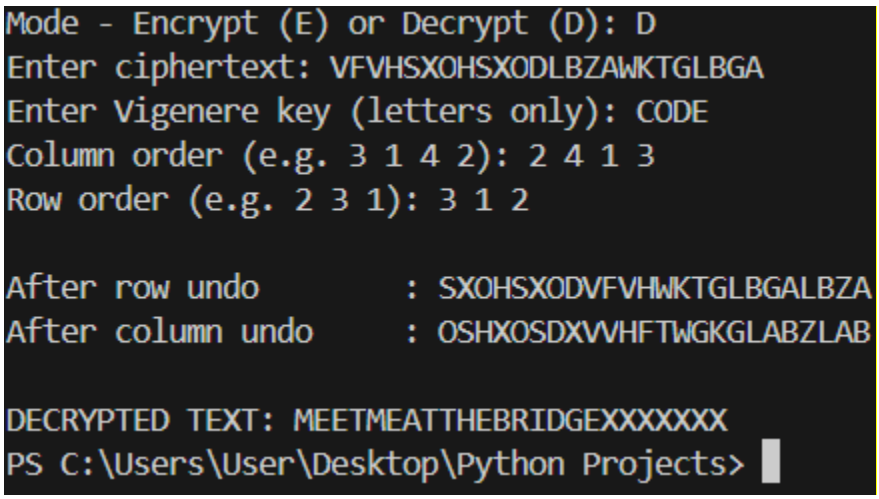
FINAL CIPHERTEXT: VFVHSXOHSXODLBZAWKTGLBGA
```

Decryption output (ciphertext pasted back in):

```
After row undo        : SXOHSXODVFVHWKTGLBGALBZA
After column undo     : OSHXOSDXVVHFTWGKGLABZLAB

DECRYPTED TEXT: MEETMEATTHEBRIDGEXXXXXXX
```

Result: The original message MEETMEATTHEBRIDGE is recovered (followed by X padding).



Test Case 3: Punctuation and mixed case

Input	Value
Mode	Encrypt
Plaintext	Hello, World!
Key	KEY
Column order	2 1 3
Row order	2 1

Encryption output:

```
After Vigenere        : RIJVSUYVJNBV
After column permute  : IRJSVUVYJBNV
After row permute     : SVUIRJBNVVYJ

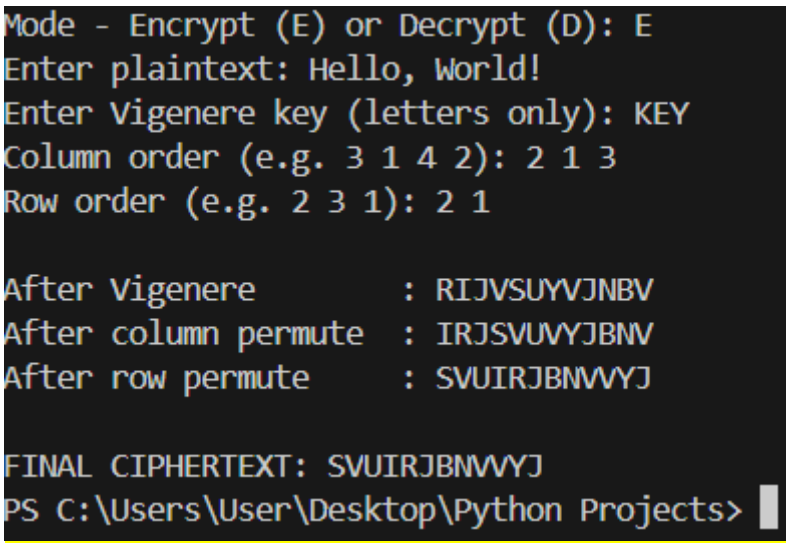
FINAL CIPHERTEXT: SVUIRJBNVVYJ
```

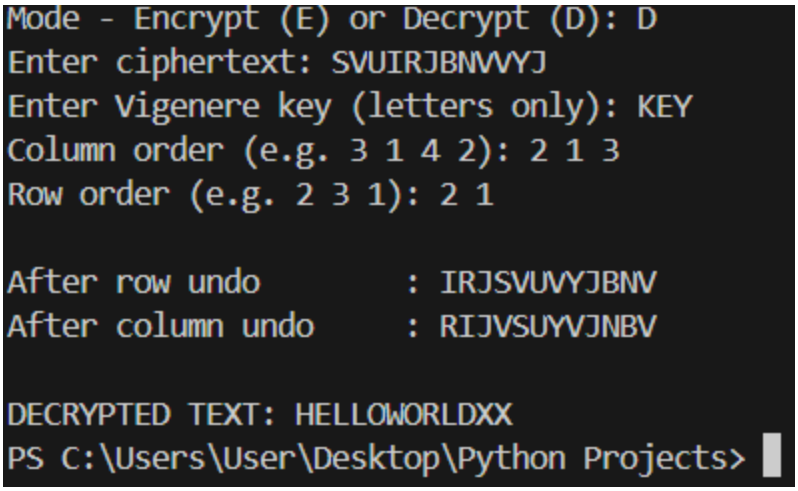
Decryption output (ciphertext pasted back in):

```
After row undo        : IRJSVUVYJBNV
After column undo     : RIJVSUYVJNBV

DECRYPTED TEXT: HELLOWORLDXX
```

Result: The comma, space and exclamation mark are removed, the letters are uppercased, and HELLOWORLD is recovered (followed by XX padding).





6. Conclusion

The program combines the Vigenère substitution with chained column and row permutations. Each step is implemented with a pair of functions that undo each other, so decryption always restores the original letters of the message. The output format is a plain string of letters, which makes copy-and-paste decryption straightforward.